



COVENANT UNIVERSITY

CANAANLAND, KM 10, IDIROKO ROAD
P.M.B 1023, OTA, OGUN STATE, NIGERIA.

B.Sc. DEGREE EXAMINATION

COLLEGE: Science and Technology

SESSION: 2024/2025

COURSE CODE: CSC241

COURSE TITLE: Python Programming Language I

INSTRUCTION: Answer Four (4) questions only

DEPARTMENT: Computer & Info. Sci

SEMESTER: OMEGA

CREDIT UNIT: 3

TIME: 3 Hours

QUESTION ONE

(a) Explain the differences among the three main types of control structures in Python. Give one example of each. [3.5mks]

(b) The following code is intended to print all even numbers from 1 to 10, skipping number 6. Identify and correct the error(s): [3.5mks]

```
i = 1
while i <= 10:
    if i % 2 == 0:
        if i == 6:
            continue
        print(i)
    i += 1
```

(c) Explain what the following code snippet does. What is the output? [3.5mks]

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("Loop finished")
```

(d) You are designing a fitness tracking application that categorises a user's activity level based on a number they enter (representing completed exercise sessions this week). Your program should use decision-making structures to implement the logic. [3.5mks]

Program requirements:

1. Prompt the user to enter an integer for the number of sessions.
2. Check if the number is positive.
3. If divisible by both 3 and 5, print "Active - FizzBuzz".
4. Else if divisible by 3, print "Active - Fizz".
5. Else if divisible by 5, print "Active - Buzz".
6. Else, print "Delulu - Not divisible by 3 or 5".
7. If the number is not positive, print "Please enter a positive number".

(e) Develop a new fitness tracking application as an extension of the one you did in (d) above. This updated version should allow the user to enter multiple session counts in one input, separated by spaces. [3.5mks]

Program requirements:

1. Prompt the user to enter multiple integers separated by spaces.
2. Split the input and convert each item to an integer.
3. For each number entered:
 - Check if it is positive.
 - Apply the decision logic.
 - Display the corresponding status.

QUESTION TWO

(a) List and briefly describe four fundamental built-in data structures in Python. For each one, mention a real-world example where it can be appropriately used. [3.5mks]

(b) The following code is supposed to store student names in a list, remove duplicates using either a 'set' or 'for loop', and print the number of unique students. Identify and fix all the issues: [3.5mks]

```
students = ["Alice", "Bob", "Alice", "Eve", "Bob"]
students = list(students)
unique_students = students
print("Total unique students:", len(unique_students))
```

(c) What will be the output of the following code? Explain what happens at each stage of execution: [3.5mks]

```
person = {
    "name": "John",
    "age": 25,
    "skills": ["Python", "Java"]
}
person["skills"].append("C++")
print(person["skills"])
```

(d) You are developing a grading support tool for teachers. The program should allow the teacher to enter the test scores of five students, store the scores in a list, sort the list in ascending order, and then display the sorted scores, highest score, and lowest score. [3.5mks]

Program requirements:

1. Initialize an empty list to hold student scores.
2. Use a loop to input the scores of five students (as integers).
3. Store each score in the list.
4. Sort the list in ascending order.
5. Print the sorted list of scores.
6. Print the maximum and minimum scores using appropriate built-in functions.

(e) Develop a new grading support tool as an extension of the one you did in (d) above. In this updated version, the program should allow the teacher to enter any number of student scores (not just five), separated by spaces. [3.5mks]

Program requirements:

1. Prompt the teacher to enter all student scores, separated by spaces.
2. Convert the input to a list of integers.
3. Sort the scores in descending order.

4. Display:

- The sorted list of scores,
- The highest and lowest scores,
- The average score,
- The number of passes and fails.

Pass mark is 45 and above

QUESTION THREE

(a) What is a function in Python? Why are functions important in programming? Differentiate between built-in functions and user-defined functions. Explain the two types of user-defined functions in Python. [3.5mks]

(b) The following code is meant to define a function that calculates the square of a number and then calls the function. However, it doesn't work as expected. Identify and fix the errors: [3.5mks]

```
def square(x)
    return x * x
print(Square(4))
```

(c) Consider the following Python code below. What will be the output of this code? Explain how the function works, especially with regard to default parameters. [3.5mks]

```
def greet(name="Guest"):
    print("Hello,", name)
greet()
greet("Ada")
```

(d) You have been asked to create a health monitoring system for a local clinic. The program should define a function `is_healthy(bmi)` that returns "Healthy" if the Body Mass Index (BMI) is between 18.5 and 24.9, and "Unhealthy" otherwise. The program should prompt the user to enter the names and BMI values of three patients, call the function for each, and display the result in the format: [3.5mks]

Patient: John | BMI: 22.4 | Status: Healthy
Patient: Ada | BMI: 17.9 | Status: Unhealthy
...

Program requirements:

1. Define the function `is_healthy(bmi)` that returns "Healthy" if $18.5 \leq \text{BMI} \leq 24.9$, else "Unhealthy".
2. In the main program, use a loop to collect each patient's name and BMI value.
3. For each patient, call the function and determine the health status.
4. Display the result in a clean, formatted output.

(e) Develop a new health monitoring system as an extension of the one you did in (d) above. This extended version should allow the clinic to enter the names and BMI values for any number of patients (not just three), all in one session. [3.5mks]

Program requirements:

1. Define the function `is_healthy(bmi)` to return "Healthy" or "Unhealthy".
2. Use a loop to repeatedly accept patient names and BMI values.
3. Stop the loop when the user enters 'done' as the name.
4. For each patient, call the function and display their status.
5. After the loop, display summary statistics.

QUESTION FOUR

(a) What is a string in Python? List and briefly explain four common string methods used in Python. [3.5mks]
Provide an example for each method.

(b) The following code is intended to extract the first name and convert it to uppercase. However, it contains errors. Identify and fix them: [3.5mks]

```
full_name = "Ada Lovelace"  
first_name = full_name[0:full_name.index()]  
print("First name:", first_name.uppercase())
```

(c) What will be the output of the following code? Explain the result of each line: [3.5mks]

```
text = " Hello, Python! "  
print(text.strip())  
print(text.lower().replace("python", "world"))  
print(len(text))
```

(d) Write a Python program for a text analysis tool that processes a list of student email addresses and prints the usernames. The username is the part before the @ symbol. [3.5mks]

Program Requirements:

1. Create a list containing at least five student email addresses.
2. Loop through the list.
3. For each email, split the string at @ and extract the username.
4. Print the username for each email in a clean format, such as:

- Email: azu.ezenwoke@covenantuniversity.edu.ng | Username: azu.ezenwoke
- Email: odunayo.osofuye@university.edu | Username: odunayo.osofuye
- Email: emmanuel.franklin@university.edu | Username: emmanuel.franklin

(e) Develop a new text analysis tool as an extension of the one you did in (d) above. This version should help a user analyze multiple sentences entered in a single paragraph. [3.5mks]

Program Requirements:

1. Prompt the user to enter a paragraph of text.
2. Split the paragraph into sentences using "." as the delimiter.
3. For each sentence:
 - Remove leading/trailing whitespace.
 - Skip empty sentences.
 - Count the number of words.
 - Convert to uppercase.
 - Count the number of vowels (a, e, i, o, u).
 - Display the results for that sentence.

QUESTION FIVE

- (a) What is file handling in Python? List and explain four file access modes used in the open() function. Also mention the difference between reading from a file and appending to a file. [3.5mks]
- (b) The following code is intended to write a message to a file named note.txt, but it contains errors. Identify and fix them. [3.5mks]

```
file = open("note.txt", "w")
file.write("This is a note.")
file.close
```

- (c) What will be the result of executing the following code? Explain what each line does [3.5mks]

```
with open("example.txt", "a") as f:
    f.write("New line added.\n")

print("Done writing.")
```

- (d) Develop a Python program for a classroom attendance system. The program should allow the teacher to enter the names of students present in class and save the list to a file called attendance.txt. [3.5mks]

Program Requirement:

1. Ask the teacher how many students were present.
2. Use a loop to input the names of each student.
3. Open a file attendance.txt in write mode.
4. Write each student's name to the file on a new line.
5. After saving, print "Attendance recorded successfully."

- (e) Develop a new student record management system as an extension of the one you did in (d) above. This enhanced version should allow the user to enter multiple student records (name and score) until 'done' is entered as the name. Each entry should be stored in a text file called "students.txt" in the format: [3.5mks]

John, 76
Ada, 45

Program Requirement:

1. Open "students.txt" in write mode to store student records.
2. Use a loop to collect student names and scores until 'done' is entered.
3. Write each entry to the file in the format: Name, Score e.g. Bethany, 18
4. Close the file.
5. Reopen the file in read mode.
6. For each line:
 - Split the name and score.
 - Print the record.
 - Count passes and fails.
 - Display total passes and fails.

Pass mark is 45 and above

Handwritten signature: *Aden*

QUESTION SIX

- (a) i. What is a database, and why is it used in programming?
ii. What role does the sqlite3 module play in Python?
iii. Mention three common SQL commands used when working with databases and their functions. [3.5mks]

(b) The following code is meant to create a database table named students, but it contains errors. Identify and fix the issues: [3.5mks]

```
import sqlite3
conn = sqlite3.connect("school.db")
cursor = conn.cursor
cursor.execute("CREATE TABLE students (id INTEGER PRIMARY KEY, name
TEXT)")
conn.commit
conn.close()
```

- (c) i. Examine the following code and explain what it does. ii. What will happen when this code runs? [3.5mks]
iii. What assumptions must be true for it to execute successfully?

```
import sqlite3
conn = sqlite3.connect("school.db")
cursor = conn.cursor()
cursor.execute("INSERT INTO students (name) VALUES ('Ada')")
conn.commit()
conn.close()
```

(d) Develop a simple Python-based contact management system using sqlite3. The program should create a database table contacts with fields: id, name, and phone. It prompts the user to enter the name and phone number of three contacts and insert each contact into the database. [3.5mks]

Program requirements:

1. Import sqlite3 and connect to a database named contacts.db.
2. Create a table contacts if it doesn't already exist.
3. Use a loop to collect name and phone number for three contacts.
4. Insert each contact into the table.
5. Commit the changes and close the connection.

(e) Develop a new contact management system as an extension of the one you did in (d) above. This extended version should allow the user to enter multiple contacts (name and phone number) until they type 'done' as the name, and, store the records in an SQLite database (contacts.db) in a table called contacts. After all entries are added, program should fetch and display all contacts stored in the table. [3.5mks]

Program Requirements:

1. Connect to contacts.db and create the contacts table if it does not exist.
2. Use a loop to input contact names and phone numbers until 'done' is entered.
3. Insert each contact into the table.
4. After the loop, select and display all contacts.